Explaining Defeasible Entailments: A Relevant Closure User Interface

Liam De Saldanha
DSLLIA002@myuct.ac.za
University of Cape Town
Cape Town, South Africa

Abstract

Defeasible reasoning is a non-monotonic reasoning approach that allows the representation of exceptional information. We use the Kraus, Lehmann and Magidor framework, an extension of propositional logic, to perform defeasible reasoning. Understanding implicit inference from defeasible information is challenging because of the complex entailment algorithms.

We propose a tool called PDR-Tool, an explanatory web debugger that lets users step through the algorithms that form part of the defeasible entailment process. By explaining the internal computations of the algorithms alongside the pseudocode definitions, we lower the barrier to entry for novices. PDR-Tool builds on prior work from previous projects and research in defeasible explainability. We discuss what the current tools for defeasible explanations offer and how we iterate on these systems.

Given a knowledge base and query, we allow users to navigate the entailment process. We incorporated 4 entailment algorithms: Rational, Lexicographical, Basic and Minimal Relevant Closure. This paper focuses on Basic and Minimal Relevant Closure. We validate the system's correctness through unit tests based on worked examples in the literature. The system's faithfulness to the entailment algorithm definitions is informally evaluated by experts.

Keywords

Relevant Closure, Defeasible Reasoning, KLM framework, Logic, Knowledge Representation and Reasoning, Pedagogical Tool

1 Introduction

Knowledge Representation and Reasoning is a field that deals with the symbolic representation of information, which we will refer to as statements. We can infer implicit information (entailments) from a collection of these statements (knowledge base) using inference procedures (entailment algorithms) [7]. The most popular logical system for representing information and performing these entailment checks is classical logic [1]. One property of classical logic is monotonicity, any inference from a subset of a set must hold in the set [14, 19]. The monotonic property of classical logic makes us unable to represent exceptions to previous statements without a cumbersome restructuring of the current knowledge base [14].

We look at a non-monotonic approach to reasoning as a solution. We consider defeasible reasoning. Defeasible reasoning allows us to infer information and revise this inference if more specific information is later added [17]. Kraus, Lehmann, and Magidor (KLM) developed a framework that provides a logical approach to defeasible reasoning [17]. We can systematically find these defeasible entailments using formalised entailment algorithms [4, 20].

Understanding entailment from a knowledge base is complex and, in the defeasible case, requires more than simply providing a list of statements. This is due to its complicated entailment procedure of ranking and removing statements [5]. In recent literature, more tools and methods in explainable defeasible entailment have been developed [5, 6, 11, 13, 18, 24]. However, a tool that provides a unified explanatory user interface and exposes the granular details of Rational Closure, Lexicographical Closure, and Relevant Closure is still needed [6, 11].

Explanation services are essential to Knowledge Representation and Reasoning. Users can debug knowledge bases by identifying which statements lead to entailments. When given a large knowledge base, it becomes difficult to see what can be inferred, even for experts [5].

This paper focuses on determining to what extent a step-through user interface faithfully visualises and explains internal computations in the Base Rank, Relevance Partition and Relevant Closure algorithms. How computationally efficient is the step-through approach, and does it maintain correctness?

The system will provide an intuitive interface so beginners can become familiar with defeasible reasoning. The implementation will visualise the construction of intermediate results. This visualisation includes Relevant Closure statement removal, finding justifications for the relevance partition, and checking statement exceptionality for base rank table construction. Explanations and correctness will be verified through informal expert feedback and unit tests based on worked examples in the literature. This interface forms part of a bigger system that incorporates Rational Closure and Lexicographical Closure explanations.

The remainder of the paper is structured as follows: Section 2 provides the necessary background to understand classical entailment and Relevant Closure. Section 3 discusses related work. Section 4 discusses the design and architecture of the web debugger. Sections 5 and 6 include the results, discussion and conclusion. An appendix is used for supplementary information such as system implementation, graphs, data and sample unit tests.

2 Background

Propositional logic forms the foundation for many other logical systems. [1]. Kraus, Lehmann and Magidor developed the KLM framework with propositional logic as the foundation. The framework formalises how we can revise conclusions from a knowledge base [11, 15–17]. In this section, we build propositional logic up to classical entailment and show how its monotonic property prevents

,
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

it from representing exceptional information. The proposed solution is non-monotonic reasoning, a closer representation of human common-sense reasoning. We define the necessary tools from the KLM framework to describe the defeasible entailment algorithm, Relevant Closure [3].

2.1 Propositional Logic

To reason about information, we represent it in a basic form that cannot be decomposed further. This is called an atom [1]. We can represent the statement "is a platypus" with atom "platypus" and "has name Perry" with atom "perry". We denote a set $\mathcal{P}$ to contain all atoms. Atoms have truth values assigned to them by a valuation function $u : \mathcal{P} \rightarrow \{T, F\}$, they are either given T or F, read as true or false, respectively. Some possible valuations for $\mathcal{P} = \{p, q, r\}$: $u_1 = \{\overline{pq}r\}$ and $u_2 = \{pq\overline{r}\}$ where $\overline{p}$ = false and p = true.

More complex information is represented by joining atoms with connectives, resulting in a formula [1]. The connectives $\rightarrow, \vee, \wedge$ are binary operators and $\neg$ is a unary operator [1]. We denote $\mathcal{L}$ to be the set of all formulas and $\mathcal{U}$ to be the set of all valuations for $\mathcal{L}$.

When an atom in a valuation is true, we say the valuation satisfies the atom. Consider a valuation u and atom p, where u satisfies p iff $u(p) = T$, denoted as $u \Vdash p$. Additionally, if $u(p) = F$ then u does not satisfy p, denoted $u \nVdash p$. This can be extended to any formula $\alpha, \beta \in \mathcal{L}$ following the criteria:

- $u \Vdash \neg\alpha$ if and only if $u \nVdash \alpha$
- $u \Vdash \alpha \wedge \beta$ if and only if $u \Vdash \alpha$ and also $u \Vdash \beta$
- $u \Vdash \alpha \vee \beta$ if and only if either $u \Vdash \alpha$ or $u \Vdash \beta$
- $u \Vdash \alpha \rightarrow \beta$ if and only if $u \nVdash \alpha$ or $u \Vdash \beta$

A knowledge base is a subset of the language where all the elements are considered to be true [11]. Consider the example: $\mathcal{K} \subset \mathcal{L}$ where $\mathcal{K} = \{p, p \rightarrow b\}$. We assert that p and $p \rightarrow b$ are true.

We can extend satisfaction to a knowledge base. We define that for a valuation u to satisfy a knowledge base $\mathcal{K}$, then for every $\alpha \in \mathcal{K}$ it must hold that $\{u \Vdash \alpha | \alpha \in \mathcal{K}\}$. We then denote this as $u \Vdash \mathcal{K}$.

If $\alpha \in \mathcal{L}$ has at least one satisfying valuation, there exists a $u \in \mathcal{U}$ such that $u \Vdash \alpha$, then we say α is satisfiable [14]. If there exists no u such that $u \Vdash \alpha$, then α is unsatisfiable. Satisfiability extends to knowledge bases. If there is at least one satisfying valuation for $\mathcal{K}$, then $\mathcal{K}$ is satisfiable. If there is no satisfying valuation for $\mathcal{K}$, then $\mathcal{K}$ is unsatisfiable.

Now we have the necessary tools to reason about what can be concluded from a knowledge base. We do this through a pattern called entailment, in this case classical entailment. We define classical entailment as: given a knowledge base $\mathcal{K}$ and formula α it is the case that α is entailed by $\mathcal{K}$ if it holds that for all $u \in \mathcal{U}$ if $u \Vdash \mathcal{K}$ then it must hold that $u \Vdash \alpha$. This can be read as α follows from $\mathcal{K}$ and is denoted as $\mathcal{K} \models \alpha$[11].

It follows that $\mathcal{K} \models \alpha$ holds, iff $\mathcal{K} \cup \{\neg\alpha\}$ is unsatisfiable. This means there is no $u \in \mathcal{U}$ such that $u \Vdash \mathcal{K} \cup \{\neg\alpha\}$. We can efficiently check the satisfiability by using a SAT solver. A SAT solver is an algorithm that checks for the existence of a satisfying valuation [1].

It is the case that once we can conclude that α follows from $\mathcal{K}$, then for any $\mathcal{K}'$ where $\mathcal{K} \subset \mathcal{K}'$, α must follow from $\mathcal{K}'$. This property is monotonicity. Humans typically reason through common sense, where we assume what we can from established knowledge and revise our conclusions when given new information. However, propositional logic cannot handle exceptions to rules. Consider the following:

Example 2.1. $\mathcal{K} = \{\text{perry} \rightarrow \text{platypus}, \text{platypus} \rightarrow \neg\text{hat}\}$ with the formula $\alpha = \{\text{perry} \rightarrow \neg\text{hat}\}$ we can conclude that α follows from $\mathcal{K}$. Now consider we learn that perry $\rightarrow$ hat and we add it to the knowledge base, this leads us to conclude both perry $\rightarrow$ hat and perry $\rightarrow \neg$hat which causes all valuations to contain $\neg$perry, meaning perry does not exist.

This is clearly undesirable and we would want to revise our conclusions. This leads us to non-monotonic reasoning as a solution.

2.2 Defeasible Reasoning

We have established that propositional logic cannot gracefully represent exceptional information. This leads us to explore non-monotonic reasoning. This paper will consider the defeasible reasoning approach. This approach draws tentative conclusions and revises them when exceptional information is added [11].

The framework extends propositional logic with the addition of the defeasible implication, a binary connective denoted as $\mid\sim$. The propositional language is extended to include the formula type $\alpha \mid\sim \beta$ which reads "if α then, normally β" [15]. A classical implication $\alpha \rightarrow \beta$ can be represented as the defeasible implication $\neg(\alpha \rightarrow \beta) \mid\sim \bot$ [14] . The new connective is incorporated into a defeasible knowledge base, an extension of the classical knowledge base. From here onwards, knowledge base refers to the defeasible variant. Similarly, instead of classical entailment $\models$ we have defeasible entailment $\mid\!\approx$. We note that it is possible to convert from a defeasible knowledge base into a classical knowledge base through a process called materialisation $\{\alpha \rightarrow \beta | \alpha \mid\sim \beta \in \mathcal{K}\}$, we denote the materialised knowledge base as $\overrightarrow{\mathcal{K}}$ [11].

We can find the exceptional information (formulas) in a knowledge base using classical entailment on the materialised knowledge base $\varepsilon(\mathcal{K}) = \{\alpha \mid\sim \beta | \overrightarrow{\mathcal{K}} \models \neg\alpha\}$[17]. The function allows us to define a sequence $\mathcal{E}_0^{\mathcal{K}}, \mathcal{E}_1^{\mathcal{K}} .. \mathcal{E}_n^{\mathcal{K}}$ where $\mathcal{E}_0^{\mathcal{K}} = \overrightarrow{\mathcal{K}}$ and $\mathcal{E}_{i+1}^{\mathcal{K}} = \varepsilon(\mathcal{E}_i^{\mathcal{K}})$. Noting that $\mathcal{E}_\infty^{\mathcal{K}} = \emptyset$ if no formulas are exceptional at every level [11]. $\mathcal{E}_\infty^{\mathcal{K}}$ contains conflicting defeasible statements, $\{\alpha \mid\sim \beta, \alpha \mid\sim \neg\beta\}$ and classical statements in $\mathcal{K}$.

Using the exceptionality sequence, we can define base rank, $br(\alpha) = \min\{r | \mathcal{E}_r^{\mathcal{K}} \not\models \neg\alpha\}$ as the smallest index of r such that a set of defeasible implications is not exceptional in $\mathcal{E}_r^{\mathcal{K}}$. We note that the base rank of a defeasible implication is the base rank of its antecedent $br(\alpha \mid\sim \beta) = br(\alpha)$. The base rank assigns a ranking over the defeasible implications, where a lower rank correlates to a higher degree of defeasibility. This ranking forms a foundational part of the defeasible entailment algorithms.

The base rank algorithm will systematically assign each formula, $\alpha \in \mathcal{K}$, the lowest possible rank while respecting exceptionality. Casini et al. formalised the high-level base rank algorithm [4]:

The output tuple from algorithm 1 contains all the statements ranked by how defeasible they are. The lower the rank, the more general and defeasible the statement is [11]. We demonstrate the base rank algorithm result through the following example:

Algorithm 1 Base Rank

Input: A defeasible knowledge base K
Output: An ordered tuple $(R_0, \ldots, R_{n-1}, R_\infty)$ and total number of ranks n

$\overrightarrow{K}$ = Materialisation$K$
$E_0 = \overrightarrow{K}$
$i = 0$
while $E_{i-1} \neq E_i$ **do**
    $E_{i+1} = \{\alpha \rightarrow \beta \in E_i \mid E_i \models \neg\alpha\}$
    $R_i = E_i \setminus E_{i+1}$
    $i = i + 1$
end while
$R_\infty = E_{i-1}$
if $R_\infty = \emptyset$ **then**
    $n = i - 1$
else
    $n = i$
end if
return $(R_0, \ldots, R_{n-1}, R_\infty, n)$

Example 2.2. Consider the following knowledge base being passed in the base rank algorithm $\mathcal{K} = \{\text{perry} \rightarrow \text{platypus}, \text{platypus} \mid\sim \neg\text{hat}, \text{perry} \mid\sim \text{hat}, \text{platypus} \mid\sim \text{bill}\}$.

$\mathcal{E}_0 = \overrightarrow{K}$
$\mathcal{E}_1 = \{\text{perry} \rightarrow \text{platypus}, \text{perry} \rightarrow \text{hat}\}$ $\mathcal{R}_0 = \{\text{platypus} \mid\sim \neg\text{hat}, \text{platypus} \mid\sim \text{bill}\}$
$\mathcal{E}_2 = \{\text{perry} \rightarrow \text{platypus}\}$ $\mathcal{R}_1 = \{\text{perry} \mid\sim \text{hat}\}$
$\mathcal{E}_3 = \{\text{perry} \rightarrow \text{platypus}\}$ $\mathcal{R}_\infty = \{\text{perry} \rightarrow \text{platypus}\}$

Rank	Formulas
R_∞	perry $\rightarrow$ platypus
R_1	perry $\mid\sim$ hat
R_0	platypus $\mid\sim \neg$hat, platypus $\mid\sim$ bill

Table 1: Base Rank Table

We can use the ordering in table 1 to determine whether a formula α is entailed by the knowledge base. This formula is known as the query. One method to check the entailments is Rational Closure. We pass the knowledge base above and the query α = perry $\mid\sim$ hat into the Rational Closure algorithm.

Algorithm 2 Rational Closure

Input: A defeasible knowledge base K and a defeasible query $\alpha \mid\sim \beta$
Output: **true** if $K \mid\approx_{RC} \alpha \mid\sim \beta$, otherwise **false**

$(R_0, \ldots, R_{n-1}, R_\infty, n) = \textsc{BaseRank}(K)$
$i = 0$
$R = \bigcup_{j=0}^{n-1} R_j$
while $R_\infty \cup R \models \neg\alpha$ **and** $R \neq \emptyset$ **do**
    $R = R \setminus R_i$
    $i = i + 1$
end while
return $R_\infty \cup R \models \alpha \rightarrow \beta$

The algorithm 2 iteratively removes all statements from a rank starting at rank 0 until the query is not exceptional in the running knowledge base ($R \cup R_\infty$). We remove all the formulas in R_0 and find that R is not exceptional. We check the final entailment $R \cup R_\infty \models \alpha$ it returns true so we conclude $\mathcal{K} \mid\approx_{RC}$ perry $\mid\sim$ hat. Rational closure is the most conservative non-monotonic entailment algorithm [11]. It is not flexible and notably weaker than other entailment algorithms from an inferential standpoint. If we were to query α = perry $\mid\sim$ bill, Rational Closure would not be able to entail the query. This is known as the drowning problem. One solution is Relevant Closure. It increases Rational Closure's inferential power [3].

This paper focuses on Relevant Closure. Rational Closure is mentioned to show the similar algorithmic structure, but the method for deciding which statements to remove differs.

Before defining the Relevant Closure algorithm, we define how to obtain the relevance partition. We note that the partition is dependent on the query antecedent. The relevance partition contains all the statements responsible for the knowledge base entailing $\neg\alpha$[3]. We use justifications to build up to the relevance partition.

We describe a justification as a subset of the knowledge base, $J \subset \mathcal{K}$. If $J \models \neg\alpha$, then for any $J' \subset J$ it holds that $J' \not\models \neg\alpha$. We can iterate the powerset of the knowledge base to find justifications.

Example 2.3. Consider the knowledge base and query from example 2.2. We find the justification: J_1 : {perry $\rightarrow$ platypus, perry $\mid\sim$ hat, platypus $\mid\sim \neg$hat}. Note that the relevance partition does not contain classical statements. We let $R = J_1$ and $R^- = \mathcal{K} - R$ [11].

A knowledge base can have more than one justification for a query. Thus, the relevance partition is defined as the union of all justifications. Using the relevance partition, we can now define the Relevant Closure algorithm 3 to demonstrate how the algorithm entails queries.

Algorithm 3 Relevant Closure

Input: A defeasible knowledge base K, a defeasible query $\alpha \mid\sim \beta$, and a relevance partition $\langle R, \bar{R}\rangle$ of K
Output: **true** if $K \mid\approx_{RelC} \alpha \mid\sim \beta$, otherwise **false**

$(R_0, \ldots, R_{n-1}, R_\infty, n) = \textsc{BaseRank}(K)$
$i = 0$
$R' = R$
while $R_\infty \cup \bar{R} \cup R' \models \neg\alpha$ **and** $R' \neq \emptyset$ **do**
    $R' = R' \setminus (R_i \cap R)$
    $i = i + 1$
end while
return $R_\infty \cup \bar{R} \cup R' \models \alpha \rightarrow \beta$

Using the knowledge base and query from example 2.2. We show that Relevant Closure algorithm 3, instead of removing entire ranks, only removes the statements in $R \cap R_i$. Meaning only statements present in the relevance partition can be removed while traversing the base ranks. In our case, only {platypus $\mid\sim \neg$hat} will be removed and {platypus $\mid\sim$ bill} remains. Hence, we can entail $\mathcal{K} \mid\approx_{RelC}$ perry $\mid\sim$ hat and $\mathcal{K} \mid\approx_{RelC}$ perry $\mid\sim$ bill. This shows how Relevant Closure is inferentially stronger. We note that Relevant Closure entails everything Rational Closure can, since it is inferentially stronger[3].

The above example depicts Basic Relevant Closure. Basic Relevant Closure is inferentially stronger than Rational Closure, but Casini et al. [3] argue that Basic Relevant Closure is still too weak and considers too many statements as relevant. Thus, Casini et al. also developed Minimal Relevant Closure. This involves the same setup as Basic Relevant Closure, but differs in how it creates the relevance partition. Instead of taking the entire justification, we only choose the lowest-ranked statement in each justification. This change causes the algorithm to remove the most defeasible information in a justification (line 5) and consider more specific information during final entailment (line 8).

Consider the justification from example 2.3. We would choose only the lowest-ranked statement in that justification. We define the minimal justification as J_1^{min} : {platypus $\mid\sim$ ¬hat}.

In this section, we describe how to obtain the Base Rank and partition for Basic Relevant Closure and Minimal Relevant Closure. We then demonstrate through an example how Basic Relevant Closure removes statements from the knowledge base to check an entailment.

3 Related Work

Systems that support defeasible entailment explanations do exist. One mechanism to explain reasoning is through justifications [13, 22]. Recent work has focused on justifications as an explanatory tool. As we analyse previous implementations of explainable defeasible entailment applications, it becomes apparent that a system that provides detailed explanations for Relevant Closure through a user interface is still needed.

Silk UI [10] is a GUI that supports justification browsing and query answering for defeasible entailments. The system uses justification exploration to debug a knowledge base and colour-code statement insertion for validation. The system assumes knowledge of defeasible reasoning and justifications, making it hard for novices to use. OntoDebug [23] is an ontology debugger. The application uses justification to aid in repairing ontologies. The interface plugs into a popular open-source ontology editor, Protege. Both SILK and OntoDebug provide systems that offer only high level explanations for entailments.

KLMDefeasibleJustificationTool [24] is a tool proposed by Wang to compute the defeasible justifications only for Rational Closure. The tool provides a textual justification as an explanation for the entailed query and which ranks have been removed. The tool does not explain the base rank construction of the knowledge base. More importantly does not handle the Relevant Closure entailment.

EXTRC [6] is a web application that provides a practical debugger for iterating through the Base Rank and Rational Closure algorithms and providing a dual modal explanation interface: mathematical and natural language. This system has no clear use of justifications to explain the final entailment and implements only Rational Closure.

Klm-algorithms [11] is a web application that provides a user interface explaining the intermediate steps through a combination of symbolic representation and a natural language interface. The web application supports Rational Closure, Lexicographical Closure and Relevant Closure. Klm-algorithms, to our knowledge, is the only system explaining defeasible entailment for Relevant Closure. However, Hamayobe indicated that each entailment algorithm would benefit from an elevated UI to improve explanations [11]. The system provides results on one scrollable page. Most explanations contain symbolic representations which make it hard to follow, since sets of formulas may look similar but contain 1 or 2 changes.

Prior work indicates that some implementations explain entailment using justifications and through natural language [6, 11]. However, a detailed user interface with granular explanatory capabilities for Relevant Closure is still needed. Our proposed solution is an explanatory step-through visualisation of the Base Rank, Partition and Relevant Closure algorithms. This provides the user with a faithful dynamic interface to explore the algorithms at a granular level. While this paper focuses on Relevant Closure, the project as a whole aims to provide a unified application for Rational Closure, Lexicographical Closure and Relevant Closure explanations.

4 System Design & User Interface

PDR-Tool is a web application. The system is separated into the frontend and backend. The backend is a REST API built using Spring Boot following a layered architecture. The frontend is implemented using React and TypeScript and communicates with the API by sending and receiving JSON objects via HTTP.

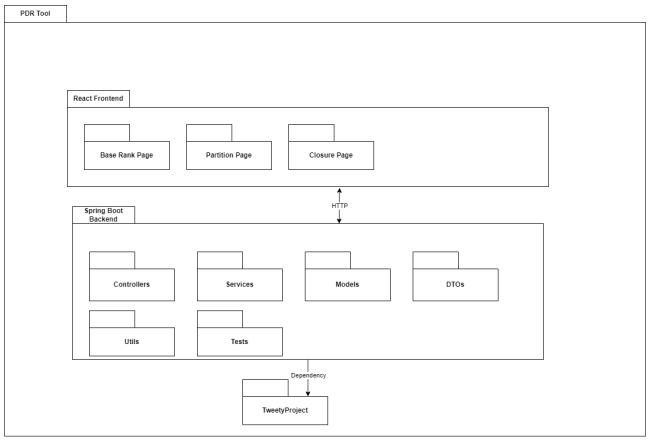


Figure 1: System Architecture

We implemented the backend in Spring Boot because of the TweetyProject. The project contains a collection of logical libraries which we use to facilitate our reasoning services. The TweetyProject contains the Sat4J SAT solver, a Java library allowing for the efficient computation of logical reasoning (satisfiability checking). We also considered how previous research had been implemented in Spring Boot or Java, and the advantage of incorporating that work [6, 11, 18, 24]. Our service layer handles the logic of our entailment algorithms, and the controller layer determines how to expose the service result over HTTP as a REST API. The Spring Boot framework makes it easy to set up and manage an API through a collection of dependencies that are auto-configured for common functions. One example is Jackson, which enables automatic JSON serialisation to and from the endpoints. The layered architecture consists of a controller and service layer. We separate the layers to reduce coupling, making it

easier to swap/add services and user interfaces. This allows future contributors the ability to easily extend the system.

We used React for our frontend because of its easy to follow component based structure. This allowed us to reuse components across the UI. React supports hot reload, which made development easier because changes are reflected at runtime. TypeScript offered type safety for easier integration with matching types across Spring Boot.

4.1 User Interface

This web application explains and visualises the steps needed to arrive at a defeasible entailment. We store intermediate computations from the algorithms that form part of the defeasible entailment process. We use a custom data structure that allows us to iterate through the computations. The data structures serve to populate iterable visualisations of the algorithms. Each iteration explains how a current step modifies a result. The result visually changes as we iterate through the data structure. Each result is clearly shown in a dedicated panel. There are different defeasible entailment algorithms. We will focus on the pages that concern the Relevant Closure algorithm. The UI consists of: a Home page, an Info page fig. 16, a Help page fig. 15, a Base Rank page, a Partition page, and a Relevant Closure page. All pages contain panels and buttons. The panels present information to the user, and the buttons navigate between pages and/or the information inside a panel. In the remainder of this section, we explain in detail what the UI does and the design decisions we made.

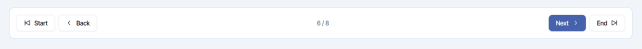


Figure 2: Navigation Controls

In order to move between iterations and pages, we need a component to control our navigation. The above fig. 2 shows the intuitive buttons that allow users to step through the visualisations either one at a time or skip to the final result. The number of iterations for a page is shown in the middle of the component. The component enables users to move backwards and forwards through the visualisation. We will showcase the remaining pages using a worked example consisting of the knowledge base from example 2.2 and query α = perry $\mid\sim$ bill. We will use Basic Relevant Closure as the entailment algorithm.

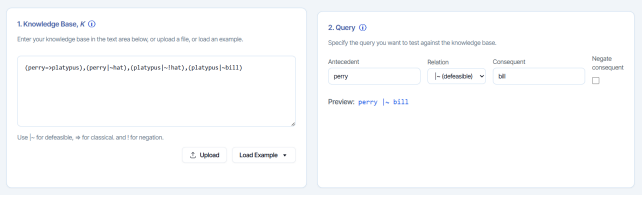


Figure 3: Knowledge Base and Query Input

The UI starts at the Home page. We enter our knowledge base and query, as seen in fig. 3. The application validates the formulas entered into the knowledge base (left panel) and indicates which formulas are incorrect. The application accepts formulas of type α => β for classical implications and $\alpha \mid\sim \beta$ for defeasible implications. The formulas allow the following connectives $\neg, \vee, \wedge$ to be used with atoms. Additionally, formulas cannot be nested. When an incorrect formula is added, the application identifies the formula, with suggestions for repair. We provide curated examples showcasing the differences between the defeasible entailments. The examples are accessible through the load example button. Custom knowledge bases can be imported through a text file. On the right panel, the UI enables quick query editing by breaking a query into its atomic components. This includes quick negation of the query consequent.

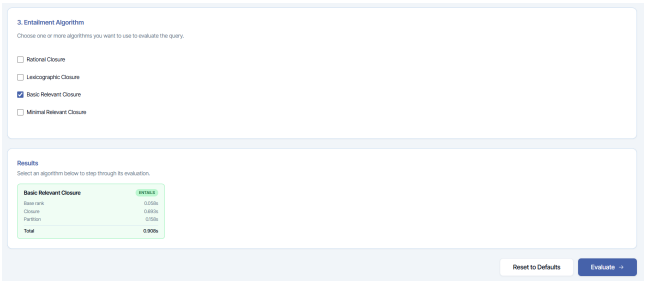


Figure 4: Entailment Algorithm Selection

After establishing our query and knowledge base, we select our entailment algorithm. We are able to choose multiple algorithms to make use of shared results and minimise redundant computation (base rank and possibly relevance partition). We select our algorithms, in this case Basic Relevant Closure and press the evaluate button. This creates dynamic buttons that indicate the entailment result, highlighted in green for entailment and red otherwise, and list the computation time for an entailment process. The UI itemises the computation time so that the performance difference between closures is clear. We created these buttons to provide quick entailment feedback and expand understanding of the algorithm's computational performance. This demonstrates how the relevance partition component of Relevant Closure affects its computation with larger knowledge bases.

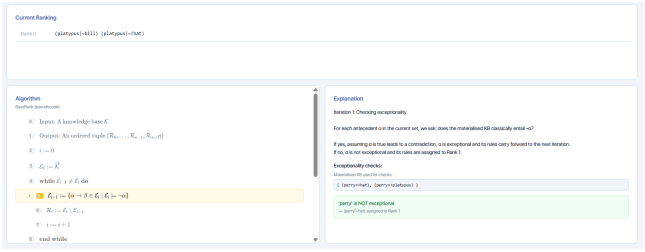


Figure 5: Base Rank Iteration

When an entailment button is clicked, we begin the explanatory step-through process. Every entailment algorithm begins with the Base Rank page fig. 5. The Current Ranking panel represents the

current base rank table, which is initially empty. The Algorithm panel represents the formalised high-level Base Rank algorithm 1 defined in pseudocode by [4]. As we iterate through the algorithm, the corresponding pseudocode line is highlighted.

The Explanation panel explains the current line in the algorithm and how it affects the base rank table in the Current Ranking panel. The majority of the pedagogical depth comes from visualising lines 4-7. Line 5 checks which statements are exceptional in the current exceptional set $\mathcal{E}_i$, to create the next exceptional set $\mathcal{E}_{i_+1}$. Exceptional statements have their antecedents coloured red, and general statements are coloured green. Line 6 assigns the statements that are green to rank i in the Current Ranking Panel. We keep performing the exceptionality check until $\mathcal{E}_i = \mathcal{E}_{i+1}$. In our example, we identified that statements with the antecedent "platypus" were not exceptional and added them to rank 0. We continued until the last defeasible implication statement was no longer exceptional. This step through approach avoids representing large amounts information at once to the user, which makes it harder to understand. Instead we explain and visualise each change iteratively one at a time.

The final result in the Current Ranking panel would reflect the table in table 1. Additionally, the Base Rank page visualises materialisation, initialisation of the first exceptional set $\mathcal{E}_0$, and how the original classical statements are assigned rank infinity $\mathcal{R}_\infty$. Once we have fully generated the Base Rank table, we can proceed to the Partition page.

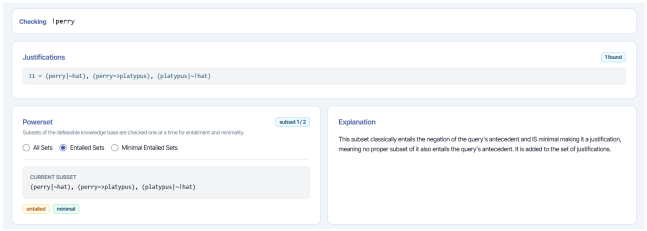


Figure 6: Relevance Partition

The Partition page consists of 3 panels: the Powerset panel, the Justification panel and the Explanation panel. The Powerset panel is analogous to the Algorithm panel in fig. 5. It represents a traversable data structure. In this context, we are iterating through the set while explaining which are justifications for the negation of the query's antecedent.

One approach to find justifications is the power set of all defeasible implications. The power set grows exponentially at a rate of 2^N, where N in this case is the number of defeasible statements. This provides an exhaustive account of possible sets, but makes it tedious to iterate through all sets. This is the most intuitive way to understand how we find justifications. Another approach for generating justifications is a hitting set tree approach implemented by Wang [12, 24]. We lose some transparency by not generating the exhaustive list, but can handle computing explanations for larger knowledge bases.

We provided additional tooling to allow users to filter the sets. Filtering addressed the gap between having to iterate through an unreasonable number of sets and skipping to the final result. The filter options include: all sets, entailed sets and minimal sets. In addition to viewing all sets, we could filter to visualise only entailed sets or visualise only justifications (entailed and minimal sets). Each filter changes the number of iterations for the Partition page. By providing filtering, we can visualise the difference between a set being entailed and being entailed and minimal. This shows the user that a set being entailed is not enough to be a justification. We show that a set is entailed and minimal with yellow and green tags in the Powerset panel, respectively.

Ultimately, to balance performance and transparency, we choose the hitting set tree approach. While traversing the power set is the most intuitive way to show how justifications are found, it generates many sets that are decorative. We would be affecting performance for sets that users wouldn't traverse. Therefore, our solution uses the hitting set tree to provide a sample of sets that entail only and all the justifications. While we cant expose sets that do not entail the negation of the query's antecedent, our work around is to append the power set of a small list of defeasible statements from the knowledge base. We would indicate to the user that the sets are not an exhaustive representation for the knowledge base but are inclusive of all justifications. We elaborate further on how each approach affected performance in the discussion 5.

In the Explanation panel, each set is explained in terms of entailing the negation of the query's antecedent and whether there exists a smaller entailing subset. When we find a set that is minimal and entailing, it is a justification. In fig. 6, it is explained to the user that this particular set entails the negation of the query and that there is no subset that also entails it. Thus, the set is a justification.

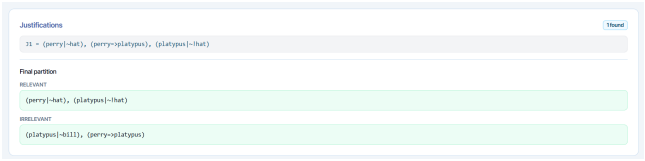


Figure 7: Relevant and Irrelevant Partition

We add justifications to a list in the Justification panel fig. 7. Once we have iterated through all the sets, we create a relevance partition that is the union of all justifications. The irrelevant partition then consists of all the remaining statements, including classical statements [11].

The above fig. 7 is a variant of the Partition page for Basic Relevant Closure, which visualises basic justifications. Minimal Relevant Closure will only show the lowest-ranked statements in each justification, forming minimal justifications. Once we build the Relevance Partition, we have all the components needed to check whether a query is in the Basic or Minimal Relevant Closure.

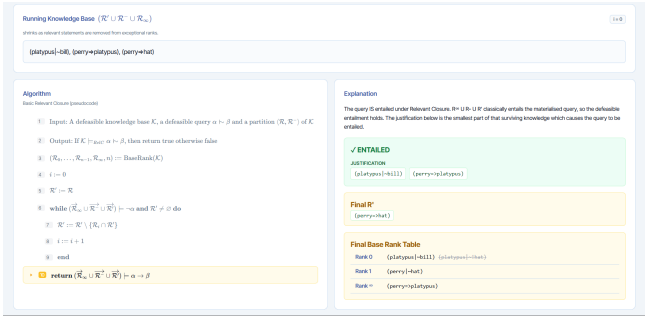


Figure 8: Relevant Closure: Final Iteration

The Relevant Closure page consists of 3 panels: an Algorithm panel, an Explanation panel and a Running Knowledge Base panel. The Algorithm panel represents the formalised high-level Relevant Closure algorithm 3 defined in pseudocode by [4]. As we iterate through the algorithm, the corresponding line is highlighted in the Algorithm panel.

The Explanation panel provides visualisations of previous results: base rank table and relevance partition. Similar to the Base Rank page, the majority of the pedagogical depth comes from explaining how a loop (lines 6-9) modifies the result. In this case, the loop removes statements from the running knowledge base, which we use to check defeasible entailments. We initially start with the full running knowledge base in the result panel, represented by "$R_\infty \cup R' \cup R^-$". When iterating through line 6, the interface provides visualisation of the statements in each set (R_∞, R', R^-) individually. This visualisation updates as we remove statements. This approach does not force the user to recall previous results. We use the union of the sets (R_∞, R', R^-), the running knowledge base, to check for entailment of the negation of the query's antecedent. This is to determine if there is still exceptional information we need to remove.

Lines 7 and 8 handle the removal visualisation for statements. Similar to line 6, the user interface provides the current visualisation of the sets making up the running knowledge base. Statements inside the set $R_i \cap R'$ are marked for removal. This is indicated by highlighting the statement in the base rank table in orange, and explicitly showing the statements in the sets $R_i \cap R'$, R'(before removal) and R'(after removal). We highlight the resulting R' in green to indicate that it is the result for this loop iteration. As we iterate through the loop and remove statements, the Running Knowledge Base panel updates.

Once we stop removing statements from the running knowledge base or when $R' = \emptyset$, we perform the final entailment check for the query. We render the final R' set, which contains any relevant statements that were not removed, and we render the final base rank table that shows which relevant statements were removed and what rank they were in. If the query was entailed, we provide the smallest weak justification responsible for its entailment. This weak justification is different from the justifications used in the Partition page and is specific to the final entailment. We provide the smallest weak justification as it contains the most specific information for the query being entailed. In fig. 8 we can see query α = perry $\not\sim$ bill is entailed along with the justifications and visualisations of which statements were removed. This step-by-step approach provides a detailed view of statement removal under Relevant Closure. To our knowledge this is the first application to show the removal process rank by rank for Relevant Closure.

The Partition, Base Rank, and Relevant Closure step-through pages all shared a similar 3-panel layout. The top panel represented a result obtained from the algorithm. The result was built incrementally as the user navigated the debugger. The bottom left panel represented the algorithm or data being traversed. The bottom-right panel represented the explanations relating to the current step in the traversal of the algorithm or element in a data structure. This provided insight into why the step modified the result. The consistent layout made it easier for users to navigate through the system.

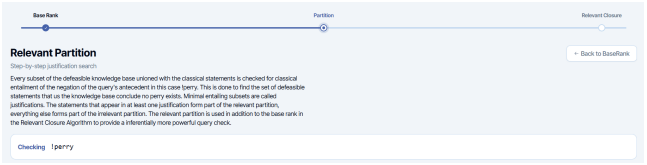


Figure 9: Progress Bar and Description

The Base Rank page, Partition page and Relevant Closure page all have a joint progress bar fig. 9. The progress bar provides the user with a visualisation of their progress during the defeasible entailment process. Each page provides a description of what the page's corresponding algorithm does. Additionally, users can use the button in the top right, to return to a previous page.

We implemented the visualisations for the above pages following Nielsen's heuristics. Visibility of system status was shown through implementation of a progress bar fig. 9 and algorithm line highlighting fig. 5, fig. 8. User freedom and control was implemented by offering users the ability to return to the home page and navigate both backwards and forwards in the step through debugger fig. 2. Consistency and Standard, all entailment algorithms followed the same 3 panel layout fig. 8, fig. 5, fig. 5, decreasing the cognitive load to find results. The Help page, offers example tutorials that address details of how to use the system [21].

To our knowledge, only one other defeasible explanatory user interface explains the Relevant Closure algorithm, klm-algorithms [11]. Our implementations differ in how thoroughly we explain and visualise each algorithm. We provide dynamic pages that guide users through constructing an algorithm's results. Klm-algorithms computes the result and returns a scrollable report to explain the results. Additionally, we provide justifications for the relevance partition and a weak justification for the final entailment check. The klm-algorithms system only returns the justifications for the entailment. Our statement removal is more intuitive and structured to follow, whereas klm-algorithms just returns the removed statements organised by base rank.

4.2 Deployment

Once the application reached a usable state, we asked experts from a research group to evaluate its faithfulness. We deployed the system on Render to make it easier to access (https://pdr-tool.onrender.com). This required creating a Dockerfile. Hosting

the application introduced performance concerns that were previously noted by [6]. This is mainly due to constraints imposed by the hosting service rather than the algorithms being inefficient. A free tier render account was used.

5 Results, Discussion & Conclusion

Throughout development, we had to ensure the correctness of the algorithms. We used Postman to check our JSON object structure. We used JUnit tests based on examples in the literature 4, and existing defeasible entailment tools to verify our implementation of the Base Rank, Partition and Relevant Closure entailment algorithms [3, 11, 20, 24]. When comparing our Basic Relevant Closure implementation with an existing implementation, we noticed there was a difference in entailments with the klm-algorithms application [11]. We resorted to a manual work through and concluded that the existing implementation was faulty:

Example 5.1. $\mathcal{K}$: $\{(p \rightarrow b), (b \mid\sim f), (b \mid\sim w), (p \mid\sim !f)\}$ α: $(p \mid\sim f)$
We construct the following base rank table:

Rank	Statements
R_∞	$p \rightarrow b$
R_1	$p \mid\sim \neg f$
R_0	$b \mid\sim f, b \mid\sim w$

Table 2: klm-algorihtms: Base Rank Table

klm-algorithm had discarded $(p \mid\sim \neg f)$ instead of correctly discarding $(b \mid\sim f)$ since it is the more defeasible statement. Interestingly when the query changed to $(p \mid\sim \neg f)$ the system correctly discarded $(b \mid\sim f)$.

We notified the researcher, and to our knowledge, this issue has been addressed. After the original fix, further testing showed another fault: the Basic Relevant Closure in klm-algorithms entailed true for the knowledge base in Table 3.8 of [11] and query α = (kittens $\mid\sim$ ¬wild) (fig. 14) instead of correctly entailing false based on the worked example in [11].

The klm-algorithm implementation correctly generated the relevance partition for Basic Relevant Closure, {(animals $\mid\sim$ wild), (cat $\mid\sim$ ¬wild), (cat $\mid\sim$ trainable), (kittens $\mid\sim$ ¬trainable)}. The problem was how the statements were then removed from R'. We are unable to provide a detailed account of the bug and why it occurs. We narrowed the problem to the method "toPowerSetOrdered" in the "BasicRelevantEntailmentBase" class and recommend that statements be removed following the formalised Relevant Closure algorithm by [4, 20]. The above motivates our decision to implement our own Relevant Closure algorithm. Designing unit tests using examples from different research papers, avoids any edge cases that one group of researchers may not have encountered with their implementations.

Our application provides explanations through internal computations. To expose this information we modified existing implementations. Our unit tests have shown that the interface has maintained correctness. We provide a sample of the test cases in the appendix, see table 4.

The explainability of our application was not formally evaluated by users, since this would have required ethics clearance, which was outside the scope of the project. Instead, our explanations and visualisation of the algorithms were informally evaluated by members of the Knowledge Representation and Reasoning (KRR) research group at the University of Cape Town. Feedback from members indicated that explanations were clear and faithful. No reasoning errors were found by members of the group. Members suggested validating formulas inserted into the knowledge base. As a result of the feedback, we reworked our input component to include real-time formula checking with descriptive error messages. This approach helps novices correct formulas and provides system control over the expressiveness of formulas.

Our application's purpose is to provide explanations for defeasible entailment. In the case of this paper, Relevant Closure explanations. The explanations were designed to be simple and intuitive. When designing the Partition page, the simplest and most exhaustive way to visualise how a justification is found would be through brute-force checking the power set. This approach was easy to understand and fit the application's theme of step-through visualisations. The limitation of this approach was the performance trade-off.



Figure 10: Execution for Defeasible Entailments for varying KB's: Power Set

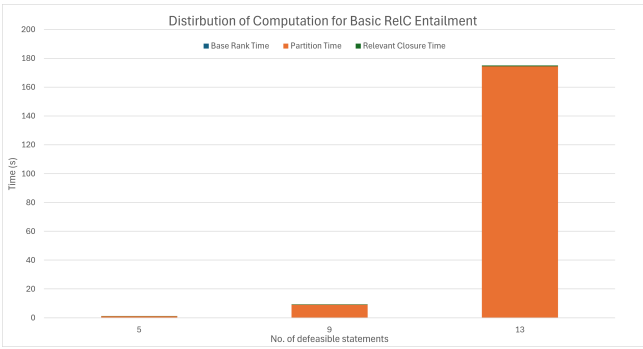


Figure 11: Distribution of Basic Relevant Closure Computation: Power Set implementation

The power set computations grew at a rate of 2^N, where N is the number of defeasible statements. This made computing explanations for larger knowledge bases impractical. We can see that the scalability of Relevant Closure is drastically worse than Rational

Closure and Lexicographical Closure fig. 10. The partition service was the main culprit fig. 11.

This performance was too drastic to justify. Our solution was to expose as much information as possible from Wang's implementation of the hitting set tree, which efficiently computes classical justifications [12, 24]. Relevant Closure performance became more competitive with the other entailment algorithms as a result fig. 12. In this implementation, instead of checking all power sets, we visualised a sample of sets, inclusive of all justifications. This approach was efficient and maintained faithfulness to explaining the relevance partition.

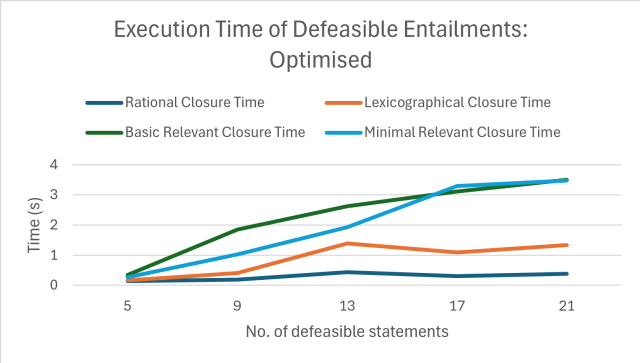


Figure 12: Execution for Defeasible Entailments for varying KB's: Hitting Set Tree

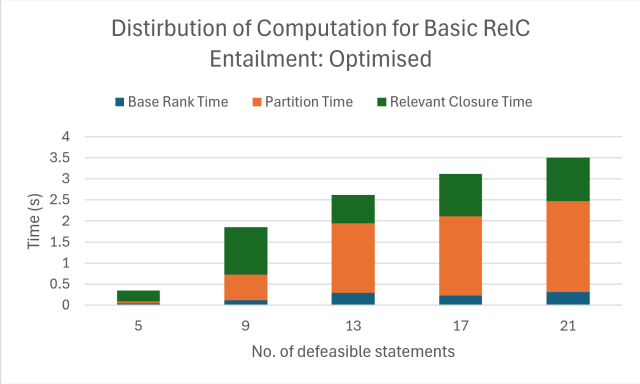


Figure 13: Distribution of Basic Relevant Closure Computation: Hitting Set Tree implementation

We can see in fig. 13 that the partition algorithm still takes most of the computation in the entailment process, as the defeasible statements in a knowledge base increases. We note that this behaviour is expected [3]. The Base Rank and Relevant Closure algorithms are, therefore, not the bottleneck. We observe that the application demonstrated the expected performance for Relevant Closure.

Our system explains 3 defeasible entailments. All entailment algorithms shared some components and pages, and as a result, were implemented as shared contributions. These shared components included: Base Rank page and implementation, Home page, Info page fig. 16 and Help page fig. 15. The Base Rank page was a collaborative design. The Home page was iterated on by all team members. On the Info page, each member included a high-level description of their algorithm and useful information and definitions for their entailment algorithms. The Help page provided an example walk through of the application.

This paper's contribution is the Relevant Closure and Partition step-by-step visualisations (pages), and algorithm implementations, which are elaborated on in the appendix section A. We designed an elevated, detailed and faithful UI with explanations and visualisations for the Relevant Closure, Base Rank and Partition algorithms. By exposing the internal computations, we generate efficient, correct and faithful explanations to users in natural language, explaining how the algorithm computed results and why entailments followed. The step through approach offers granular explanations, which does not obscure the final result or overload the user with symbolic representations. The transparency of the application leaves no guessing about why a statement is removed from an entailment algorithm or why a set is not a justification. Our application makes it easier for beginners to become familiar with algorithms in defeasible reasoning.

In this paper, we have explored work done in explainable defeasible entailment. We concluded that a detailed user interface was needed to elevate explanatory services for the Relevant Closure entailment process. We proposed PDR-Tool as a solution. It is a step-through debugger that visualises the Base Rank, Partition and Relevant Closure algorithm. We implemented the system in Spring Boot with a React and TypeScript frontend. We verified its correctness through examples in the literature and existing defeasible reasoning tools. In comparison to existing explanatory defeasible entailment interfaces, our application provides an elevated UI with granular natural language explanations and visualisations relating to the entailment process.

Future contributors could add strong justifications to the defeasible entailment explanations to provide additional insight. Strong justifications always entail the query no matter what statements from the knowledge base are added to the justification [9]. There exists a formalised algorithm for computing strong justifications for Rational Closure [9]. We are unaware of a formalised algorithm to compute strong justification for Relevant Closure and do not know how difficult formalising one would be.

A another approach to justification is minimal rank establishing sets (MRES) [13]. This would show why a set of statements can be regarded as most specific when computing a justification. There currently exists only a Rational Closure formalisation.

Future work that does not involve formalising algorithms includes, implementing justification browsing [10, 23] and converting textual representations of justifications into visual trees/graphs [2].

More ambitious work can consider the integration of AI in defeasible explainability. LLM's have already been used to generate more interpretable knowledge bases [8]. One potential usage is an AI assistant to explore slightly modified knowledge bases without leaving the original step-through explanation. To ensure correctness, the AI would use defined methods in the application through Spring

AI. This makes the AI's explanation traceable to implemented algorithms. The AI does not need to be bound to simply returning text.

6 AI Acknowledgement

Grammarly was used to correct grammatical errors in the paper. This did not involve generative AI. Grammarly's reader reaction feature, which I believe uses generative AI, was used to identify potential questions a reader might have that the paper did not answer. Claude Sonnet 5, was used to provide critical feedback. No content was generated, and all thoughts are the authors' own.

References

[1] Mordechai Ben-Ari. 2012. *Mathematical Logic for Computer Science.* Springer, New York. doi:10.1007/978-1-4471-4129-7

[2] Philippe Besnard and Sylvie Doutre. 2004. Checking the acceptability of a set of arguments. *NMR*, 59–64. https://www.researchgate.net/publication/221535800_Checking_the_acceptability_of_a_set_of_arguments

[3] Giovanni Casini, Thomas Meyer, Kodylan Moodley, and Riku Nortjé. 2014. Relevant Closure: A New Form of Defeasible Reasoning for Description Logics. In *Logics in Artificial Intelligence*, Eduardo Fermé and João Leite (Eds.). Springer International Publishing, Cham, 92–106. doi:10.1007/978-3-319-11558-0_7

[4] Giovanni Casini, Thomas Meyer, and Ivan Varzinczak. 2019. Taking Defeasible Entailment Beyond Rational Closure. In *Logics in Artificial Intelligence*, Francesco Calimeri, Nicola Leone, and Marco Manna (Eds.). Springer International Publishing, Cham, 182–197. doi:10.1007/978-3-030-19570-0_12

[5] Victoria Chama. 2020. *Explanation for defeasible entailment.* Master's thesis. University of Cape Town. http://hdl.handle.net/11427/32206

[6] Julia Cotterrell. 2025. *A User Interface to Aid in the Understanding of Rational Closure Entailments.* BSc (Honours) Project. University of Cape Town. https://projects.cs.uct.ac.za/honsproj/cgi-bin/view/2025/cotterrell_dunn_gounden.zip/#resources

[7] James P. Delgrande, Birte Glimm, Thomas Meyer, Miroslaw Truszczynski, and Frank Wolter. 2023. Current and Future Challenges in Knowledge Representation and Reasoning. arXiv:2308.04161 [cs.AI]. doi:10.48550/arXiv.2308.04161

[8] Jethro Dunn. 2025. *Implementing Natural Language Into Knowledge Base Generation.* BSc (Honours) Project. University of Cape Town. https://projects.cs.uct.ac.za/honsproj/cgi-bin/view/2025/cotterrell_dunn_gounden.zip/#resources

[9] Lloyd Everett, Emily Morris, and Thomas Meyer. 2022. Explanation for KLM-Style Defeasible Reasoning. In *Artificial Intelligence Research*, Edgar Jembere, Aurona J. Gerber, Serestina Viriri, and Anban Pillay (Eds.). Vol. 1551. Springer International Publishing, Cham, 192–207. Series Title: Communications in Computer and Information Science. doi:10.1007/978-3-030-95070-5_13

[10] Benjamin Grosof, Mark Burstein, Mike Dean, Carl Andersen, Brett Benyo, William Ferguson, Daniela Inclezan, and Richard Shapiro. 2010. A SILK graphical UI for defeasible reasoning, with a biology causal process example. In *Proceedings of the 2010 International Conference on Posters & Demonstrations Track - Volume 658* (Shanghai, China) *(ISWC-PD'10)*. CEUR-WS.org, Aachen, DEU, 113–116. https://www.researchgate.net/publication/221466022_A_SILK_Graphical_UI_for_Defeasible_Reasoning_with_a_Biology_Causal_Process_Example

[11] C. Hamayobe. 2026. *Implementation and Evaluation of KLM-Style Defeasible Entailment and Explanation Algorithms.* Master's thesis. University of Cape Town.

[12] Matthew Horridge. 2011. *Justification Based Explanation in Ontologies.* Ph. D. Dissertation. The University of Manchester. https://research.manchester.ac.uk/en/studentTheses/justification-based-explanation-in-ontologies/

[13] Jane Imrie. 2025. *Justifications for KLM-style defeasible reasoning.* Master's thesis. University of Cape Town. http://hdl.handle.net/11427/42351

[14] A. Kaliski. 2020. *An Overview of KLM-Style Defeasible Entailment.* Master's thesis. University of Cape Town, Cape Town, South Africa. http://hdl.handle.net/11427/32743

[15] Sarit Kraus, Daniel Lehmann, and Menachem Magidor. 1990. Nonmonotonic reasoning, preferential models and cumulative logics. *Artificial Intelligence* 44, 1 (1990), 167–207. doi:10.1016/0004-3702(90)90101-5

[16] Daniel Lehmann. 1995. Another perspective on default reasoning. *Annals of Mathematics and Artificial Intelligence* 15, 1 (March 1995), 61–82. doi:10.1007/BF01535841

[17] Daniel Lehmann and Menachem Magidor. 1992. What does a conditional knowledge base entail? *Artificial Intelligence* 55, 1 (1992), 1–60. doi:10.1016/0004-3702(92)90041-U

[18] Thabo Vincent Moloi. 2024. *Extending Defeasible Reasoning Beyond Rational Closure.* BSc (Honours) Project. University of Cape Town. https://projects.cs.uct.ac.za/honsproj/cgi-bin/view/2024/mohlerepe_moloi_sadiki.zip/resources/EXTRC_Final_Paper_MLXTHA036.pdf

[19] J. Donald Monk. 1986. The Contributions of Alfred Tarski to Algebraic Logic. *The Journal of Symbolic Logic* 51, 4 (1986), 899–906. doi:10.2307/2273903

[20] Matthew Morris, Tala Ross, and Thomas Meyer. 2020. Algorithmic definitions for KLM-style defeasible disjunctive Datalog. *South African Computer Journal* 32, 2 (Dec. 2020). doi:10.18489/sacj.v32i2.846

[21] Jakob Nielsen and Robert Mack. 1994. Usability Inspection Methods: Book by Jakob Nielsen. https://www.nngroup.com/books/usability-inspection-methods/

[22] Bijan Parsia, Evren Sirin, and Aditya Kalyanpur. 2005. Debugging OWL ontologies. In *Proceedings of the 14th International Conference on World Wide Web* (Chiba, Japan) *(WWW '05)*. Association for Computing Machinery, New York, NY, USA, 633–640. doi:10.1145/1060745.1060837

[23] Konstantin Schekotihin, Patrick Rodler, and Wolfgang Schmid. 2018. OntoDebug: Interactive Ontology Debugging Plug-in for Protégé. In *Foundations of Information and Knowledge Systems*, Flavio Ferrarotti and Stefan Woltran (Eds.). Springer International Publishing, Cham, 340–359. doi:10.1007/978-3-319-90050-6_19

[24] Steve Wang. 2023. *Defeasible justification for the KLM Framework.* Master's thesis. University of Cape Town. http://hdl.handle.net/11427/39922

A Supplementary Information

This paper focuses on implementing 4 algorithms to explain defeasible entailment in the KLM framework. These algorithms are Base Rank, Partition, Basic and Minimal Relevant Closure.

The Base Rank implementation is adapted from prior work [6, 18]. The adaptations included structuring the Base Rank model and creating the BaseRankStep model, which represents one iteration of the Base Rank algorithm. BaseRankStep contains which statements are exceptional, which are not exceptional and which are in the current rank. This adaptation was a shared responsibility among all members. The Base Rank service was adapted to include the tracer BaseRankStep. Since Spring Boot handles serialisation automatically through Jackson, we had to create a DTO (Data Transfer Object) for the Base Rank model, since Jackson would keep going down the object hierarchy until it reached a primitive. This would result in a large, unreadable JSON object with an unknown structure. The DTO would define what information the controller exposed. To avoid an explosion of recursive key-value pairs, we converted knowledge bases and other objects to a string representation.

Relevant Closure had only one prior implementation, which was by Hamayobe [11]. We implemented our own Basic and Minimal Relevant Closure entailment algorithm and used Hamayobe's Relevant Closure implementation to test correctness.

We started by creating the Partition service. Similar to the Base Rank service, we created the Partition and PartitionStep models to hold intermediate states of the algorithm and DTO's. The service used the Wang's [24] compute all classical justification implementation, forming candidate sets. We exposed sets after an Expand and Contract algorithm was applied. We iterate through all candidate sets, finding which sets made the query antecedent exceptional. We flag them as justifications. If they are also minimal, we flag that as well. We use the PartitionStep data structure , which contains flags indicating minimality and entailment, and the list of all justifications so far. When the iteration finishes, we join all justifications to form the relevant and irrelevant partitions and store them in the Partition object. This process describes creating the partition for Basic Relevant Closure. For Minimal Relevant Closure, we would need to further process the candidate sets justifications and find the lowest-ranked statement in every justification.

The Relevant Closure service follows a standard similar to Base Rank and Partition by defining models for RelevantEntailment, Entailment, EntailmentStep and the DTO's. Because the system considers multiple entailment algorithms, we implement the entailment model as an interface, allowing each entailment algorithm to implement the service logic. Following [20] high-level definition of Relevant Closure in propositional logic, we implement the Relevant Closure service. We use the builder pattern to include information and easily extend the RelevantEntailment class.

A.1 Unit Test Examples

Data was obtained on a laptop with the following specification: 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz and 16GB DDR4 RAM. The gap in computation time between Basic and Minimal Relevant Closure in table 7 and table 8 can be attributed to Basic Relevant Closure running first then Minimal Closure running when we press the evaluate button. This causes the Basic Relevant Closure to load all the relevant data into memory (cold run) and when Minimal Relevant Closure runs it makes use of the already loaded memory (warm run). We ran warm up iterations before recording the result, but the pattern was still present.

A.2 User Interface

Number of defeasible statements	Knowledge Base	Query
5	$(c1 \mid\sim wild), (c1 \mid\sim q1), (c1 \mid\sim r1), (c1 \mid\sim s1), (c1 \mid\sim t1)$	$(c1 \mid\sim !wild)$
9	$(c1 \mid\sim wild), (c1 \mid\sim q1), (c1 \mid\sim r1), (c1 \mid\sim s1), (c1 \mid\sim t1), (c2 \rightarrow c1), (c2 \mid\sim !wild), (c2 \mid\sim q2), (c2 \mid\sim r2), (c2 \mid\sim s2)$	$(c2 \mid\sim !wild)$
13	$(c1 \mid\sim wild), (c1 \mid\sim q1), (c1 \mid\sim r1), (c1 \mid\sim s1), (c1 \mid\sim t1), (c2 \rightarrow c1), (c2 \mid\sim !wild), (c2 \mid\sim q2), (c2 \mid\sim r2), (c2 \mid\sim s2), (c3 \rightarrow c2), (c3 \mid\sim wild), (c3 \mid\sim q3), (c3 \mid\sim r3), (c3 \mid\sim s3)$	$(c3 \mid\sim !wild)$
17	$(c1 \mid\sim wild), (c1 \mid\sim q1), (c1 \mid\sim r1), (c1 \mid\sim s1), (c1 \mid\sim t1), (c2 \rightarrow c1), (c2 \mid\sim !wild), (c2 \mid\sim q2), (c2 \mid\sim r2), (c2 \mid\sim s2), (c3 \rightarrow c2), (c3 \mid\sim wild), (c3 \mid\sim q3), (c3 \mid\sim r3), (c3 \mid\sim s3), (c4 \rightarrow c3), (c4 \mid\sim !wild), (c4 \mid\sim q4), (c4 \mid\sim r4), (c4 \mid\sim s4)$	$(c4 \mid\sim !wild)$
21	$(c1 \mid\sim wild), (c1 \mid\sim q1), (c1 \mid\sim r1), (c1 \mid\sim s1), (c1 \mid\sim t1), (c2 \rightarrow c1), (c2 \mid\sim !wild), (c2 \mid\sim q2), (c2 \mid\sim r2), (c2 \mid\sim s2), (c3 \rightarrow c2), (c3 \mid\sim wild), (c3 \mid\sim q3), (c3 \mid\sim r3), (c3 \mid\sim s3), (c5 \rightarrow c4), (c5 \mid\sim wild), (c5 \mid\sim q5), (c5 \mid\sim r5), (c5 \mid\sim s5)$	$(c5 \mid\sim !wild)$

Table 3: Knowledge Bases used for stress testing application

Author	Knowledge Base	Query	Defeasible Entailment Algorithm	Entailed in literature	Entailed in our application
Steve Wang	(penguins $\rightarrow$ birds), (robins $\rightarrow$ birds), (specialpenguins $\rightarrow$ penguins), (birds $\mid\sim$ fly), (birds $\mid\sim$ wings), (penguins $\mid\sim$ $\neg$fly), (specialpenguins $\mid\sim$ fly)	(specialpenguins $\mid\sim$ fly)	Minimal Relevant Closure	TRUE	TRUE
Chipo Hamayobe	(pets $\rightarrow$ animals), (kittens $\rightarrow$ cats), (cats $\mid\sim$ trainable), (kittens $\mid\sim$ $\neg$trainable), (animals $\mid\sim$ legs), (animals $\mid\sim$ wild), (cats $\rightarrow$ animals), (cats $\mid\sim$ $\neg$wild)	(kittens $\mid\sim$ $\neg$wild)	Minimal Relevant Closure	TRUE	TRUE
Chipo Hamayobe	(pets $\rightarrow$ animals), (kittens $\rightarrow$ cats), (cats $\mid\sim$ trainable), (kittens $\mid\sim$ $\neg$trainable), (animals $\mid\sim$ legs), (animals $\mid\sim$ wild), (cats $\rightarrow$ animals), (cats $\mid\sim$ $\neg$wild)	(kittens $\mid\sim$ $\neg$wild)	Basic Relevant Closure	FALSE	FALSE

Table 4: Unit tests for Relevant Closure

$\mathcal{K} = \{ (pets \rightarrow animals), (cats \rightarrow animals), (kittens \rightarrow cats), (kittens \mid\sim \neg trainable), (cats \mid\sim \neg wild), (cats \mid\sim trainable), (animals \mid\sim wild), (animals \mid\sim legs) \}$

$\alpha = (kittens \mid\sim \neg wild)$

Algorithm Results

Inference Operator	Entailment	Justification
Rational Closure	$\mathcal{K} \not\approx (kittens \mid\sim \neg wild)$	$\mathcal{J}_1 = \{\}$
Lexicographic Closure	$\mathcal{K} \approx (kittens \mid\sim \neg wild)$	$\mathcal{J}_1 = \{(kittens \rightarrow cats), (cats \mid\sim \neg wild)\}$
Basic Relevant Closure	$\mathcal{K} \approx (kittens \mid\sim \neg wild)$	$\mathcal{J}_1 = \{(kittens \rightarrow cats), (cats \mid\sim \neg wild)\}$
Minimal Relevant Closure	$\mathcal{K} \approx (kittens \mid\sim \neg wild)$	$\mathcal{J}_1 = \{(kittens \rightarrow cats), (cats \mid\sim \neg wild)\}$

Figure 14: Klm-algorithms: Entailment Check for kittens $\mid\sim$ $\neg$wild

# Defeasible Statements	Base Rank	Partition	Relevant Closure
5	0.035	1.101	0.186
9	0.075	9.026	0.295
13	0.149	174.351	0.599

Table 5: Distribution of computation in seconds for Basic Relevant Closure: Power Set implementation

# Defeasible Statements	Base Rank	Partition	Relevant Closure
5	0.041	0.046	0.261
9	0.117	0.605	1.131
13	0.292	1.654	0.671
17	0.228	1.884	1.001
21	0.308	2.165	1.032

Table 6: Distribution of computation in seconds for Basic Relevant Closure: Hitting Set Tree implementation

# Defeasible Statements	Rational Closure	Lexicographical Closure	Basic Relevant Closure	Minimal Relevant Closure
5	0.092	0.101	1.322	1.164
9	0.123	0.294	9.396	9.572
13	0.293	0.840	175.099	149.879

Table 7: Computation time of all entailments in seconds for 3 different knowledge bases: Power Set implementation

# Defeasible Statements	Rational Closure	Lexicographical Closure	Basic Relevant Closure	Minimal Relevant Closure
5	0.14	0.169	0.348	0.263
9	0.184	0.402	1.853	1.02
13	0.442	1.394	2.617	1.933
17	0.308	1.093	3.113	3.287
21	0.377	1.329	3.505	3.478

Table 8: Computation time of all entailments in seconds for 5 different knowledge bases: Hitting Set Tree implementation

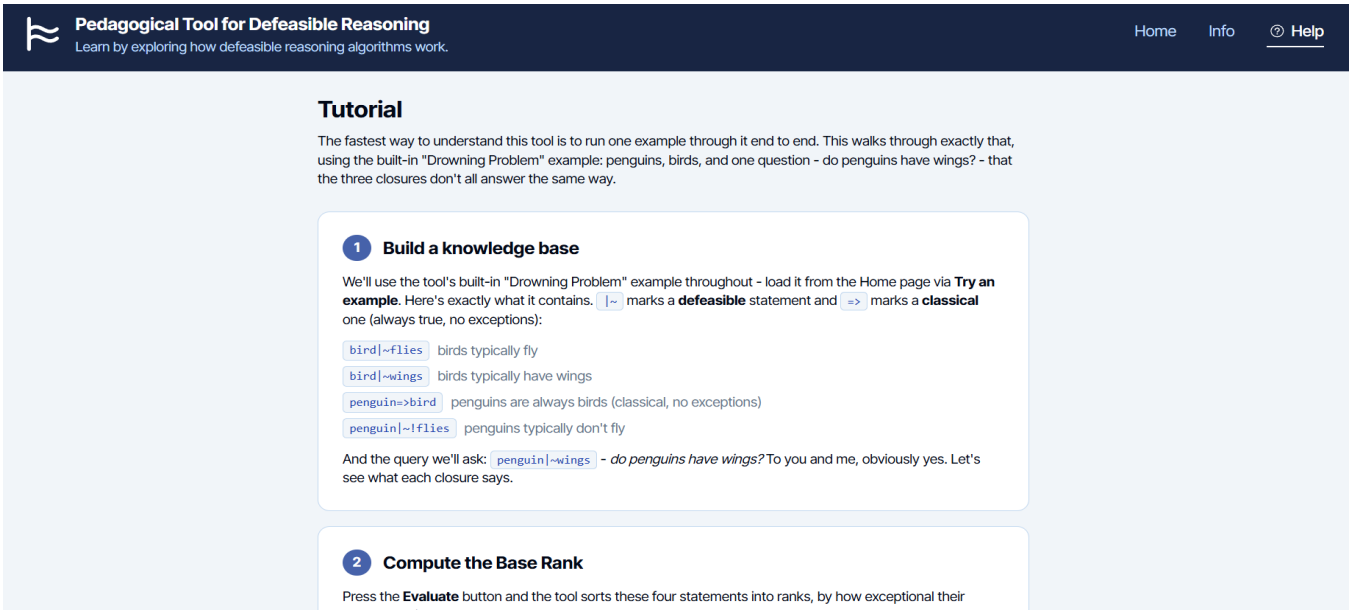


Figure 15: Help Page

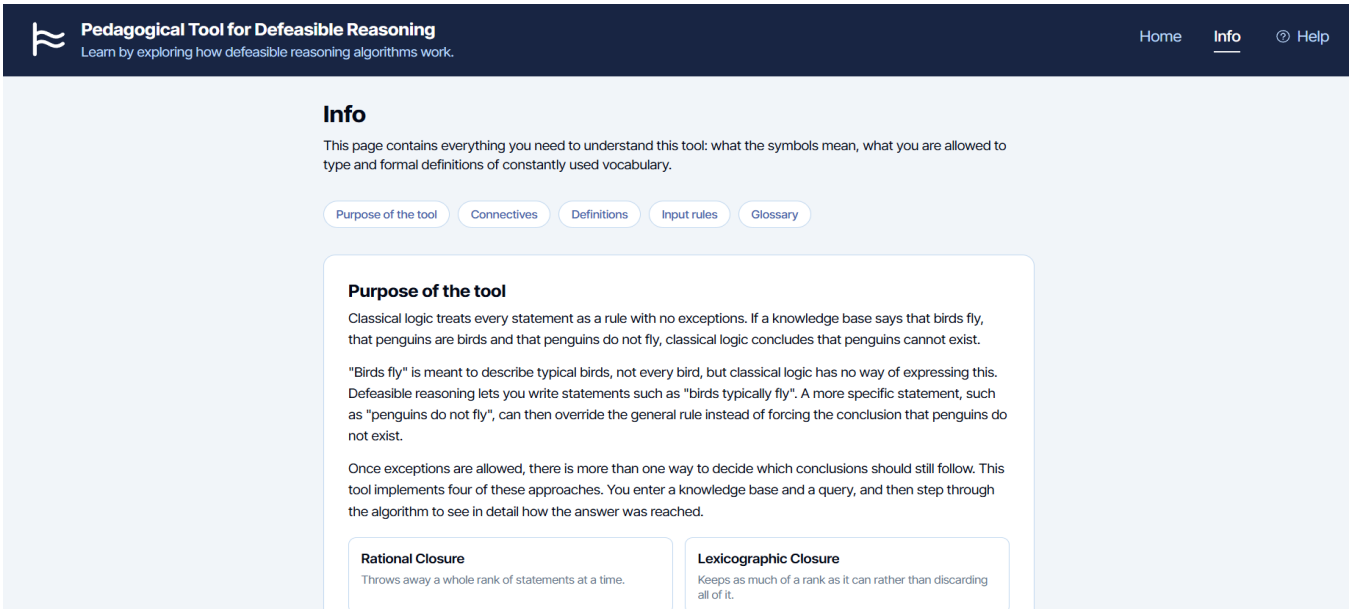


Figure 16: Info Page